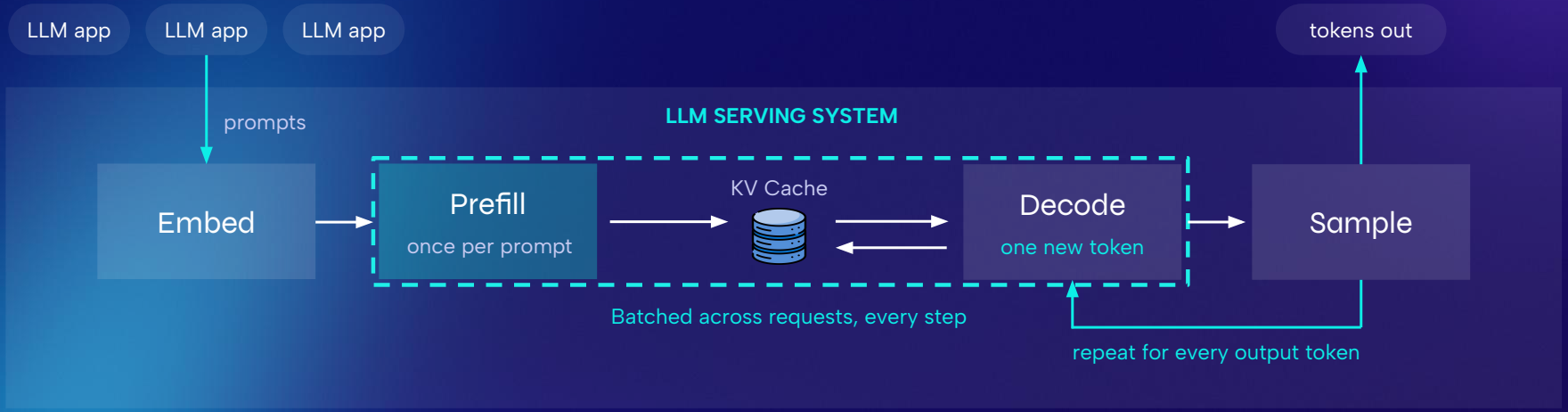
Pie

A Programmable Serving System for Emerging LLM Applications

In Gim, Zhiyao Ma, Seung-seob Lee, Lin Zhong
Yale University
SOSP '25, Seoul, October 2025

Presented by: Dhruv Rawat (dhruvr), Toby Buckfire (tqbuck), Hongkwon Kim (hongkwon), Donna Pham (phdonn)

How LLM serving works today



1 Prefill

Process the whole prompt in parallel. Fill the KV cache once.

2 Decode

Predict one token. Append to the KV cache. Repeat.

Batch

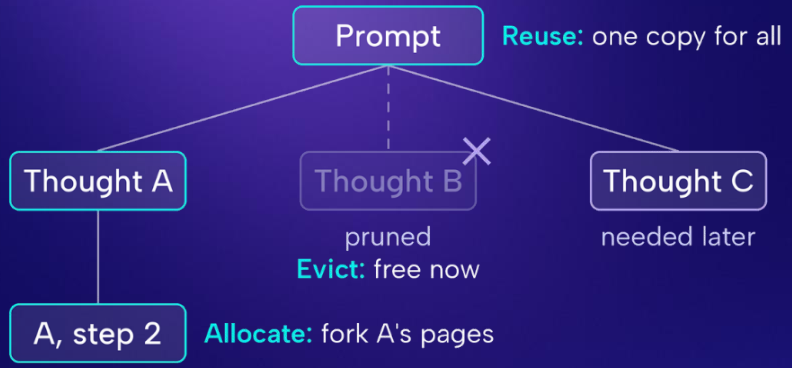
One scheduler steps many requests together under global KV policies.

Three requirements for next-gen serving

R1 Application-specific KV cache control

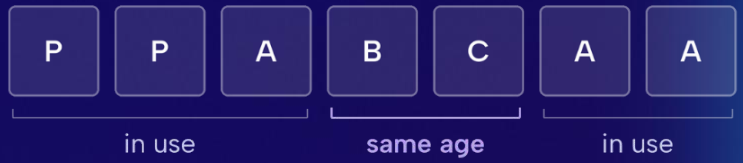
Tree of Thoughts explores branches, prunes the weak ones, and backtracks to the old ones.

What the app knows



What the server sees

KV pages on the GPU



LRU ranks pages by last use alone.

Dead B and needed C look the same.

What is missing Servers apply one global policy (eg. LRU or prefix caching). The app has no say

Three requirements for next-gen serving

R2 Customizable generation

Each request wants its own way to pick the next token, even step by step

What each request needs

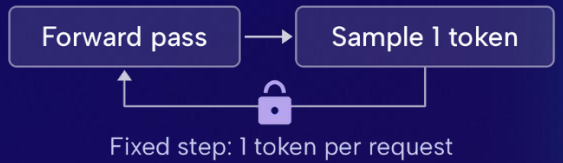
Speculative decoding
guess 4 tokens, check in one pass

JSON grammar
block tokens the schema forbids

Watermarking
favor a new secret set each step



What the server offers



Speculative decoding: on for all, or none

Each new method needs engine changes.

What is missing One fixed predict-then-sample step, tuned by server-wide switches

Three requirements for next-gen serving

R3 Integrated compute and I/O

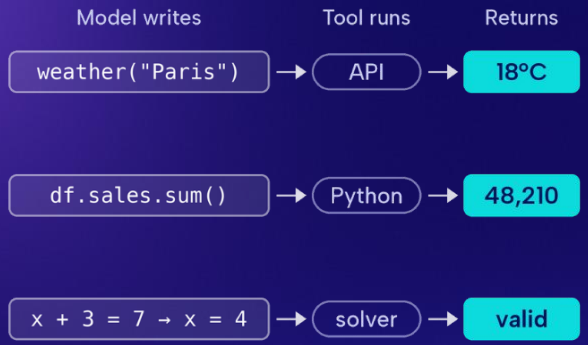
Agents pause mid-answer to call tools, then continue with the result

What the agent needs

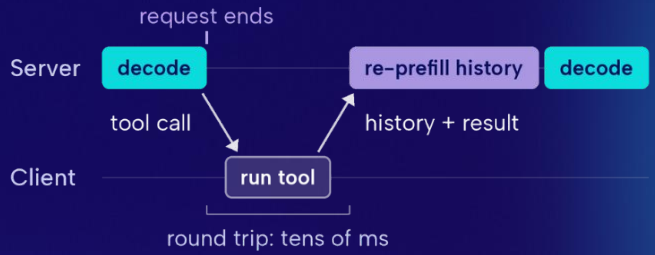
Call an API
fetch live data mid-answer

Run code
compute, then read the output

Check with a solver
verify a step before going on



What the server offers



Prefill is skipped only on a prefix cache hit.

Repeats on every call: 8 to 32 per task.

What is missing Tools run outside the loop. Every call ends the request and starts a new one

Design Principle: Serve programs, not prompts

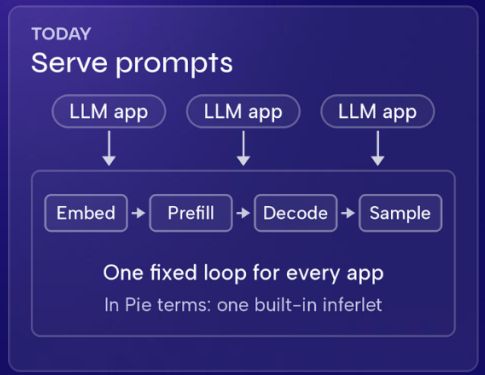
Root cause: App control logic is tightly coupled to the engine. Pie separates them with two principles

1 Split the loop into an API

Every loop step, such as embed, forward or KV page allocation, becomes a small API call. Pie offers 42.

2 Hand control to inferlets

An inferlet is a user program, compiled to WebAssembly, running inside the server and calling the API.





Benefit: New cache policies, decoders and agent loops ship as programs, with no server changes.

Embed, forward, sample

01 Embed

Turn text or images into input embeddings.

`embed_txt` `embed_img`

02 Forward

Run the transformer. Read and write KV pages, with an optional mask.

`forward` `mask_kvpage`

03 Sample

Read next-token scores. Your code picks the token.

`get_next_dist`

Two resources pass between stages by pointer. You allocate and free both.

Embed one token embedding, allocated per token

KvPage 8 to 32 tokens of KV cache (PagedAttention)

Command queue

Orders your API calls, marks dependencies and sets priority. The scheduler batches calls when batching is safe.

42 APIs. 18 for the forward pass and memory, 24 for runtime and I/O.

API in Action: A decode loop you write yourself

```
prom = tokenize("Hello, ")
prom_emb = alloc_emb(len(prom))
gen_emb = alloc_emb(1)
kv = alloc_kvpage(tok_limit)
```

```
embed_txt(prom, pos, prom_emb)
forward([], prom_emb, kv[:n], gen_emb)
```

```
for i in range(n, tok_limit):
    dist = await get_next_dist(gen_emb)
    gen = dist.max_index()
    embed_txt(gen, [i], gen_emb)
    forward(kv[:i], gen_emb, kv[i:i+1], gen_emb)
```

```
dealloc_emb(prom_emb)
dealloc_kvpage(kv)
```

1 Allocate

You own embeddings and KV pages.

2 Prefill

One forward call fills the KV pages.

3 Decode

You read the distribution and pick the token. Swap in any sampler.

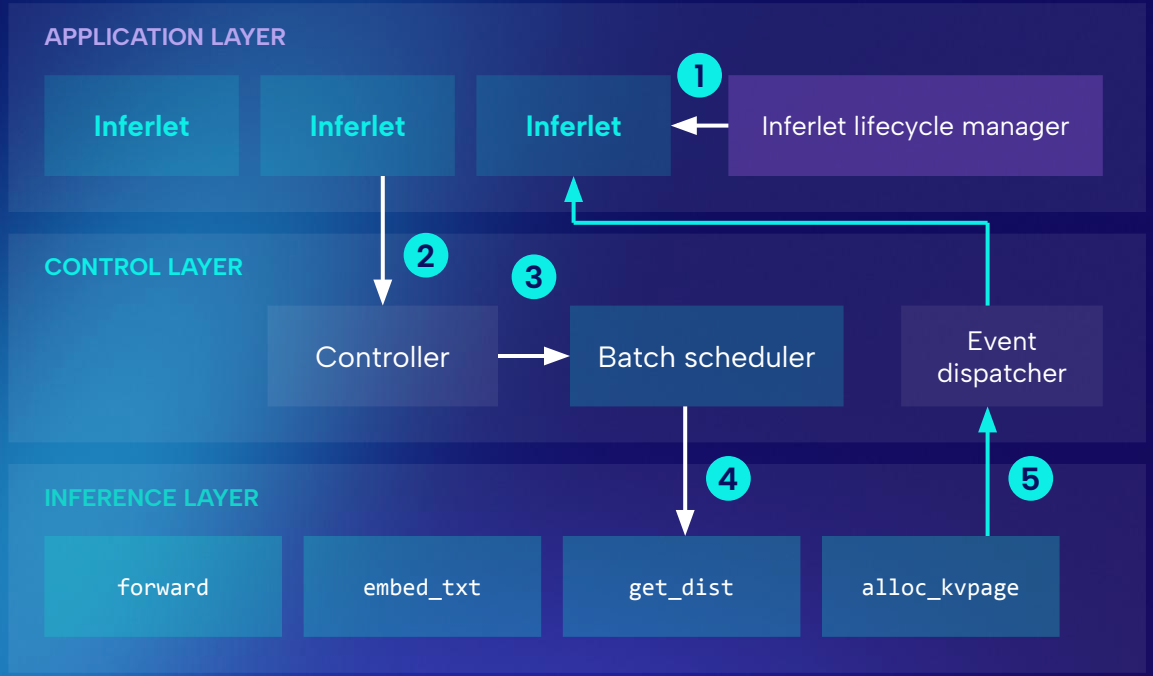
4 Free

Release resources explicitly.

```
ctx = Context(model)
ctx.fill("Hello, ")
ctx.generate_until(max_tokens=10)
```

Support library: the same loop in three lines. Think OpenGL for LLM inference. Low-level control for experts, libraries for everyone else.

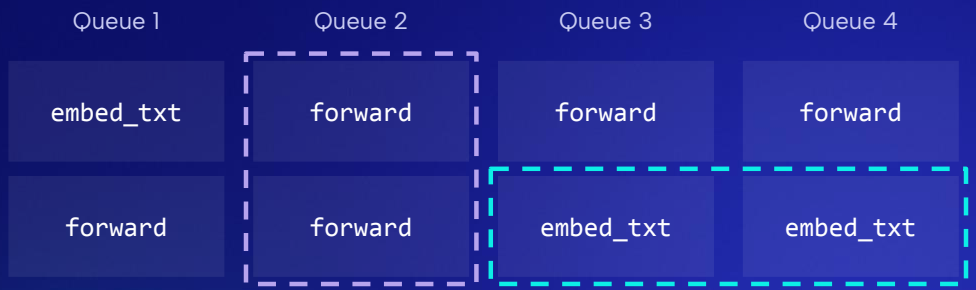
Three layers, one request path



1. **Launch** The client submits a Wasm binary through the lifecycle manager.
2. **Call** Inferlets issue API calls. Non-GPU calls finish in the control layer.
3. **Batch** The scheduler groups GPU calls from many inferlets.
4. **Execute** Handlers run batched kernels on the GPU.
5. **Return** The event dispatcher sends results back to each inferlet.

13,650 lines. Rust core, Python GPU handlers, wasmtime, FlashInfer.

Batching without a fixed loop



Top to bottom: order within each queue.

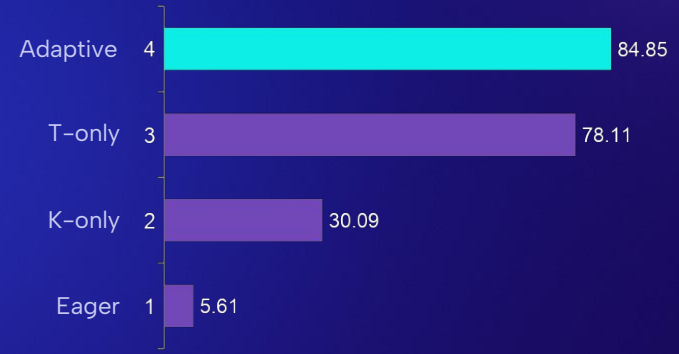
Horizontal batching Same call type across queues: the two embed_txt calls.

Vertical batching Consecutive same-type calls in one queue, if they do not conflict: the two forward calls in queue 2.

Higher-priority queues go earlier in a batch.

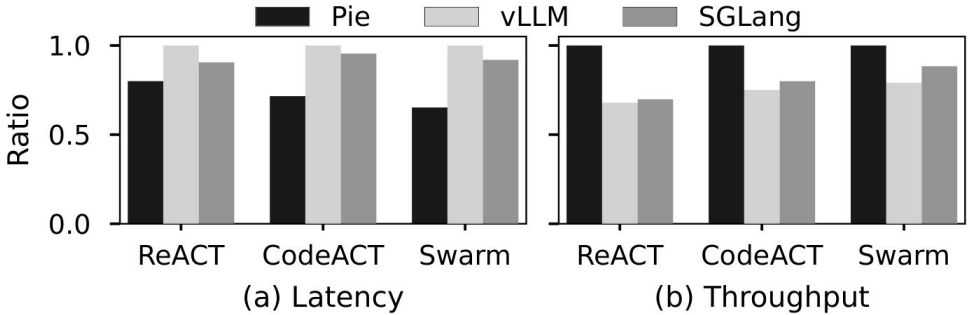
Work-conserving dispatch

GPU busy: queue incoming calls. GPU idle: form a batch and dispatch now.



Requests per second with 128 concurrent inferlets (paper Table 5).

Results: agentic workflows



Paper Figure 6. Latency (lower is better) and throughput (higher is better), normalized to the worst system in each case.

Setup

One NVIDIA L4 GPU, Llama 3 1B for agents. 8 tool calls per ReACT and CodeACT agent, 32 per Swarm agent. Baselines: Python clients on vLLM 0.6.0 and SGLang 0.4.4, all on FlashInfer kernels.

15% Lower latency on ReACT, best case.

30% Higher throughput on ReACT, best case.

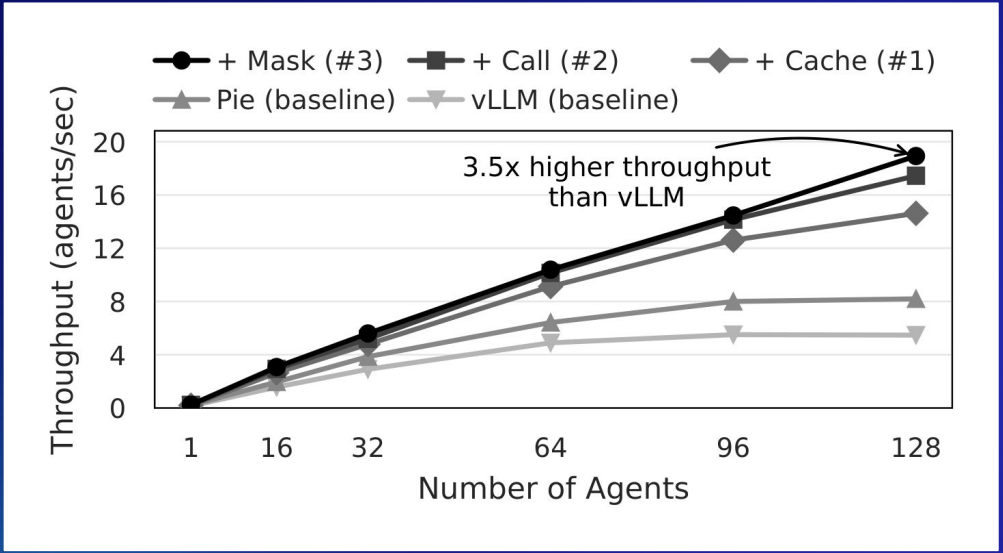
Why Pie wins

1B and 3B: no client round trip per tool call.

8B and up: cached KV skips re-prefill.

More tool calls widen the gap. Under two calls, no gain.

Results: workload-specific tuning



Paper Figure 7. Each optimization stacks on the previous one.

3.5× higher throughput than the vLLM baseline at 128 agents

Assume three traits of the workload. Pie lets you act on each one.

1

Hot APIs

Some APIs get used most. Keep their doc KV pages with export_kvpage.

2

Fire and forget

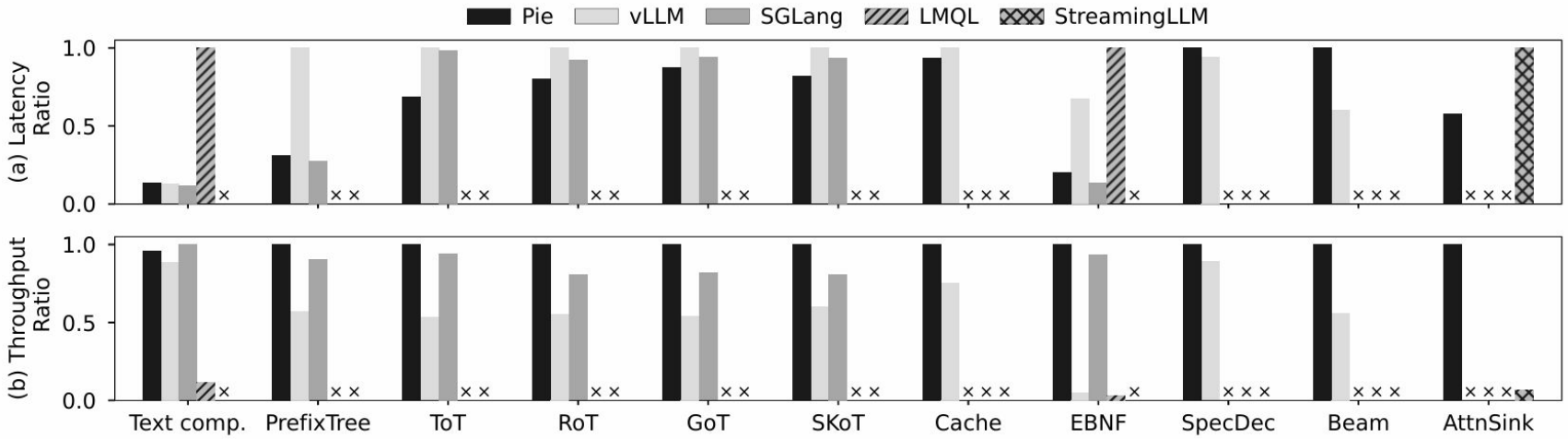
Most calls need no reply. Launch each one when its signature appears.

3

Used once

Most specs appear once. Drop their KV pages with mask_kvpage.

Results: reasoning and parity



Paper Figure 8. Crosses mark unsupported techniques or impossible comparisons.

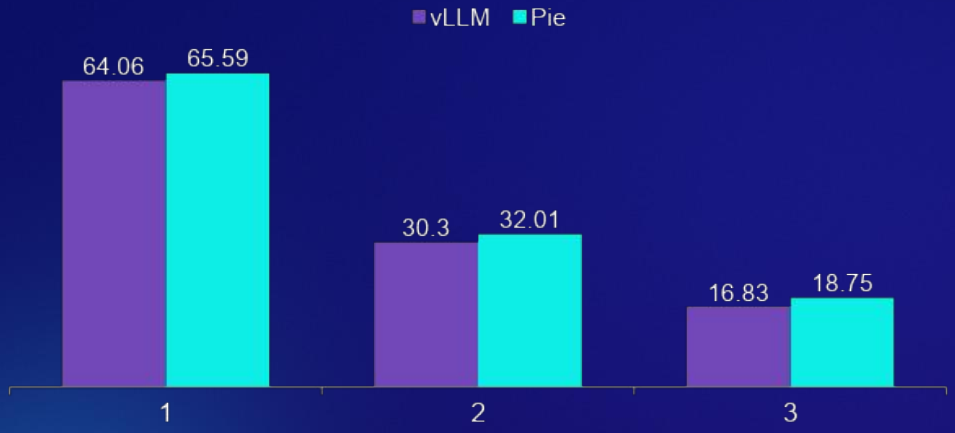
28% Up to 28% lower latency, 34% higher throughput on ToT, RoT, GoT and SKoT.

30× Over 30× throughput against StreamingLLM on attention sink. Kernels differ.

Parity with vLLM and SGLang on caching, speculative decoding, beam search, EBNF.

What programmability costs

Time per output token (ms) on text completion, one L4 GPU.



+2.4%
8B model, +1.53 ms

+5.6%
3B model, +1.71 ms

+11.4%
1B model, +1.92 ms

Where the 1.53 ms goes (8B)



- Lost GPU pipelining of sampling
- 0.2 ms: input embedding, batching, distribution return, layer crossings, Wasm

Launch 10 to 50 ms warm, 35 to 81 ms cold, with 896 inferlets at once.

API call Under 30 µs in the control layer. 10 to 300 µs through the Python inference layer.

Batching The adaptive policy beats eager, fixed-size and timeout policies.

How Pie differs from related works

vLLM and TGI run one fixed loop. LangChain, DSPy and AutoGen sit above serving systems and stay orthogonal.

CONSTRAINED DECODING

LMQL, Guidance, XGrammar

Controls the shape of the output, via grammars and constraints

Stops at inside the standard decoding loop.

PROGRAMMATIC SERVING

SGLang, Parrot

Controls prompt structure and workflows. Shared prefixes, fork and join.

Stops at same monolithic loop. No page-level KV control or in-loop I/O.

WASM CONTROLLERS

AICI

Controls token-level steering with Wasm programs.

Stops at what to generate, not how inference runs.

PIE

Programmable inference

Controls KV pages, the embed, forward and sample steps, tokenization and I/O.

Cache policies such as Pensieve and HCache become user programs.

Trade-off a low-level API. More work per application.

Depth of user control over generation

Strengths and weaknesses

Strengths

Expressive
19 techniques as inferlets, 38 to 255 lines each, from Tree of Thought to ReAct agents.

Low overhead
2% to 11% per token. The cost shrinks as models grow.

Cheap isolation
Wasm sandbox. Launches in 10 to 81 ms. Hundreds of inferlets run at once.

Composable and open
Third-party libraries drop in, such as llguidance. Traits keep the API stable. Open source.

Weaknesses

Narrow evaluation
One 24 GB L4 GPU, Llama 3 up to 8B, one node. Scale-out is untested.

Hand-tuned wins
The 3.5× needs expert tuning for one workload. Baselines are Python clients on older vLLM and SGLang.

Developer burden
Low-level API with manual allocation. Easy to leak pages or break correctness.

Security and fairness
Untrusted code, network I/O and logit access widen the attack surface. FCFS kills the newest inferlet.

Where to go next

Scale out

Multi-node control layer. Place and migrate KV pages. Enforce SLOs across GPUs.

Fuse APIs

Fuse forward and sample automatically. Win back the 1.3 ms lost to pipelining.

Smarter policies

Quotas, priority-aware preemption, and CPU and GPU swap of KV pages.

Harden security

Capability-based I/O, rate limits, and capped or obscured logits.

Better front ends

Compile SGLang or DSPy programs into inferlets. Verify inferlets before they run.

Takeaways

01 Fixed serving loops limit KV control, decoding and tool I/O.

02 Pie turns generation into API calls. Wasm inferlets drive them on the GPU.

03 The price: 2% to 11% per token. The gain: up to 3.5× on agentic workloads.

Whole paper in one sentence: **Serve programs, not prompts.**

Paper: Gim et al., SOSP '25 (CC BY 4.0). Code: github.com/pie-project/pie.

Murakkab: Resource-Efficient Agentic Workflow Orchestration in Cloud Platforms

Gohar Irfan Chaudhry[1], Esha Choukse[2], Haoran Qiu[2], Íñigo Goiri[2], Rodrigo Fonseca[2], Adam Belay[1], Ricardo Bianchini[3]

[1] MIT CSAIL [2] Microsoft Azure Research – Systems [3] Microsoft Azure

arXiv:2508.18298v2, September 2025

UNIVERSITY OF MICHIGAN

Introduction

What is Murakkab?

A system that coordinates the models, tools, and computing resources used by an AI workflow.

Example: answering a question about a video

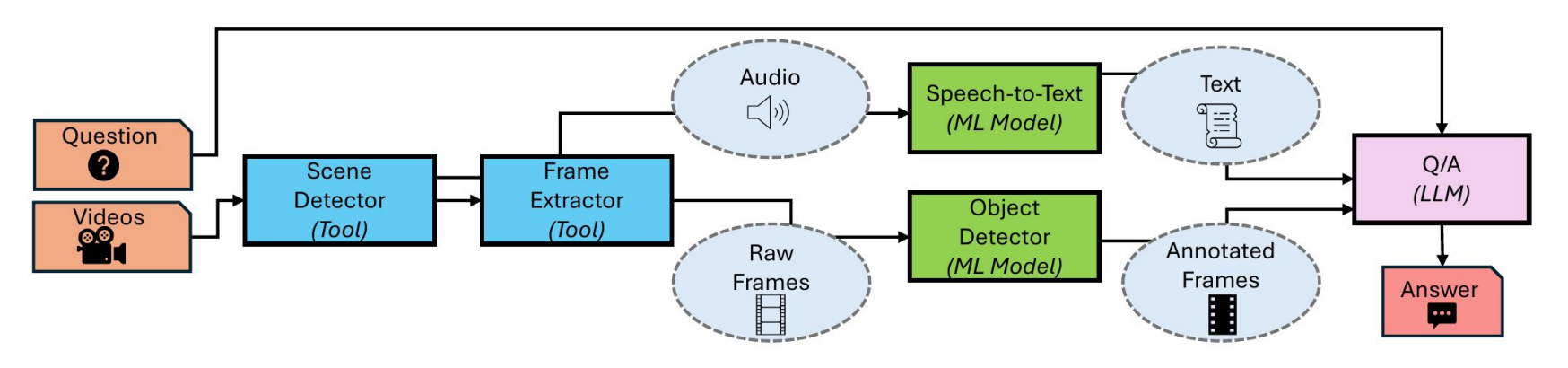
Extract frames, transcribe speech, then run a language model.

The goal

Use fewer GPUs and reduce energy or cost while meeting the user's accuracy and response-time requirements.

An example workflow

Video question answering



Each stage has its own model, tool, and resource needs.

Motivation: rigid choices

The same workflow has many configurations

Workflow	How many frames? Which stages can run together?
Model	Which model is sufficient for this request?
Hardware	Which GPU type and degree of parallelism?

Motivation: fragmentation

Different layers see different parts of the problem

User	Accuracy target and response deadline
Developer	Workflow logic and model choices
Cloud platform	Available GPUs and current load

Why joint optimization is necessary

Section 2.5: quality, workload, and hardware interact

Workflow + model	Higher-quality configurations can generate many more tokens.
Hardware	GPU type and parallelism change latency and energy efficiency.
Joint choice	The best feasible configuration depends on SLOs, load, and resources.

Source: Murakkab §2.5, Figures 3–5. Manual tuning cannot cover this joint space.

Problem statement

How can we reduce resource use while meeting each request's quality and latency targets?

SLO = service-level objective

Configuration must adapt to demand and available hardware.

Murakkab's approach

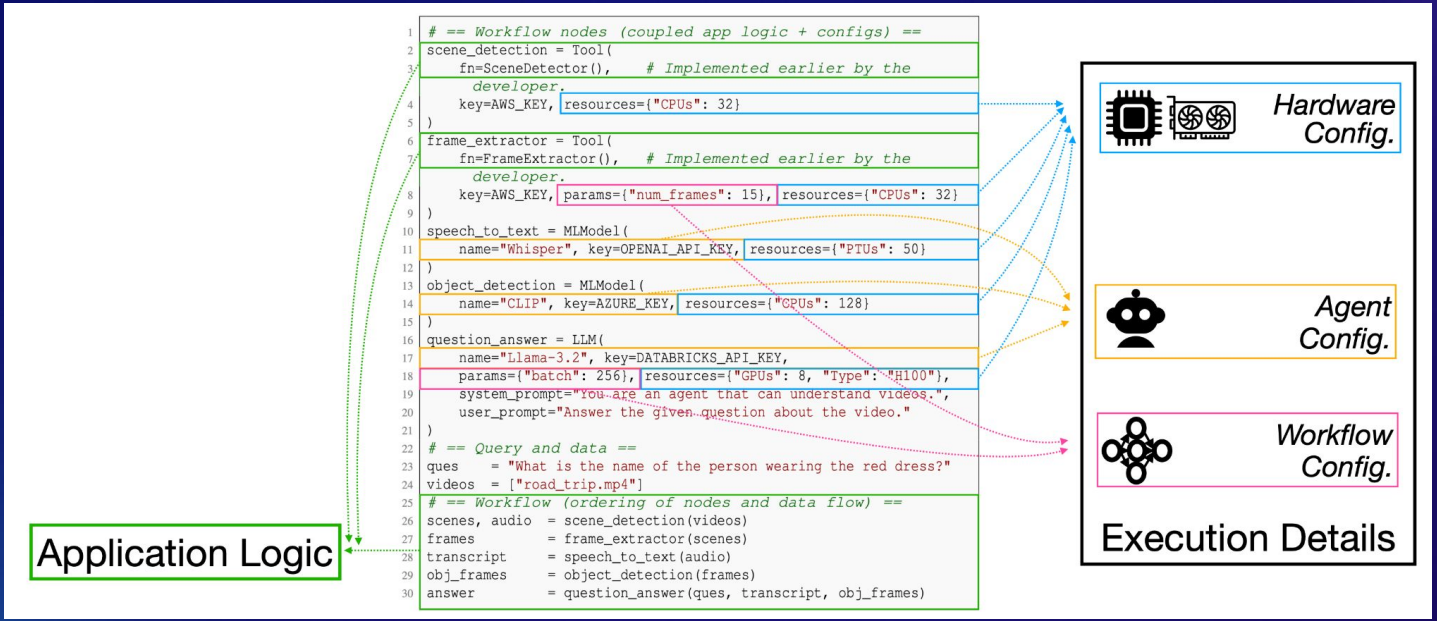
1. Separate workflow logic from execution settings

Developers specify tasks, dependencies, and user targets.

2. Adapt execution at runtime

The system chooses models and resources for current demand.

Methodology



Divide the application logic and execution details while Maintaining a track visibility

Methodology

Declarative Workflow Specification

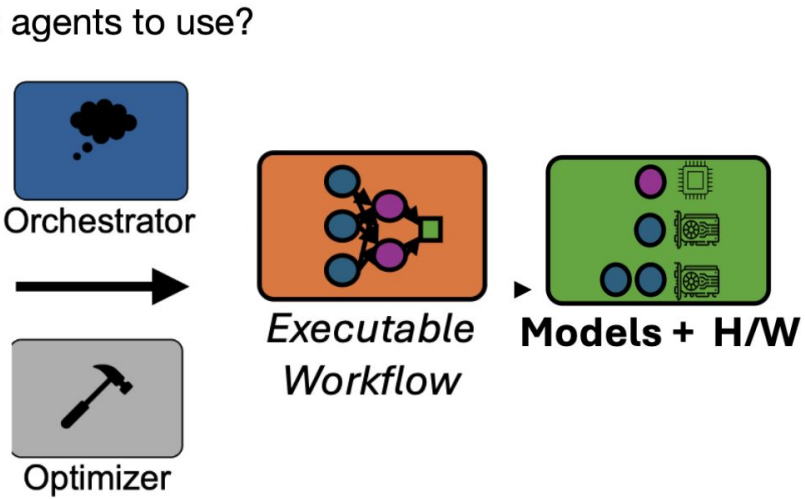
```
# == Sub-tasks in the workflow ==
scene_detect   = "Given a list of videos, identify scenes in each."
frame_extract  = "Given a list of scenes, extract frames."
stt            = "Given a list of scenes, convert audio to text."
q_a            = "Answer the query given some context."
# == Workflow description (sub-tasks and data flow) ==
def workflow(query, videos):
    scenes      = scene_detect(videos)
    frames      = frame_extract(scenes)
    transcript  = stt(scenes)
    answer      = q_a(query, [frames, transcript])
    return answer
# == Execution with example request ==
query   = "What is the name of the person wearing the red dress?"
videos  = ["road_trip.mp4"]
result  = run(workflow(query, videos), slo=LOW_LATENCY)
```

Separate the "what" from the "how".

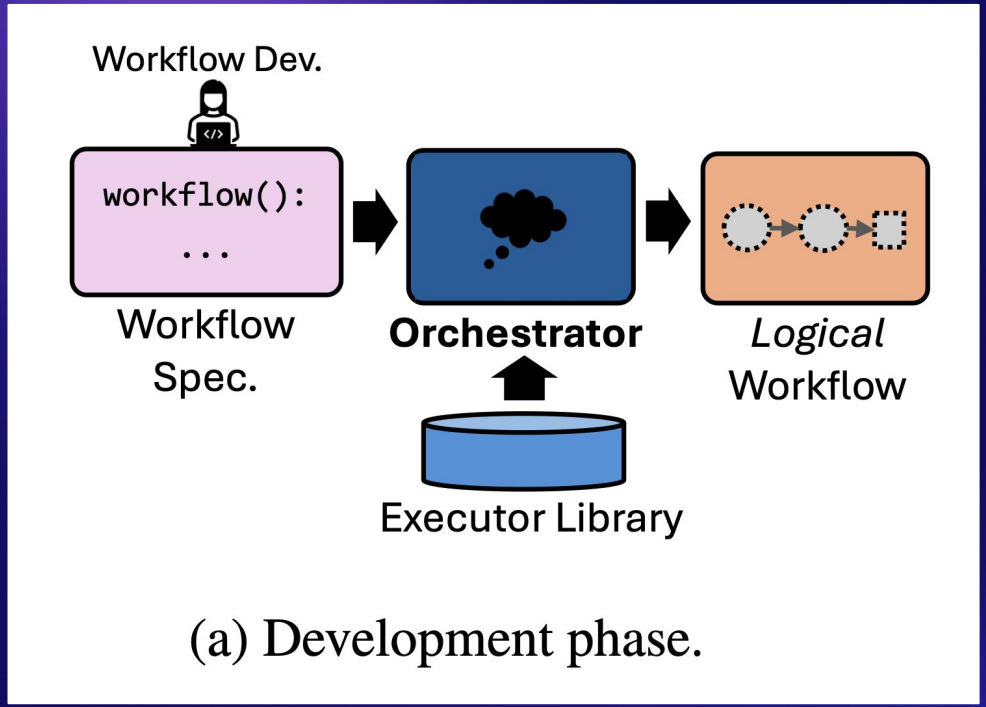
- Re-define workflows
- Define configuration using SLO (Service Level Objective)
- Receive executors (functions) as string

Methodology

```
# == Sub-tasks in the workflow ==
scene_detect   = "Given a list of videos, identify scenes in each."
frame_extract  = "Given a list of scenes, extract frames."
stt            = "Given a list of scenes, convert audio to text."
q_a            = "Answer the query given some context."
# == Workflow description (sub-tasks and data flow) ==
def workflow(query, videos):
    scenes      = scene_detect(videos)
    frames      = frame_extract(scenes)
    transcript  = stt(scenes)
    answer      = q_a(query, [frames, transcript])
    return answer
# == Execution with example request ==
query   = "What is the name of the person wearing the red dress?"
videos  = ["road_trip.mp4"]
result  = run(workflow(query, videos), slo=LOW_LATENCY)
```

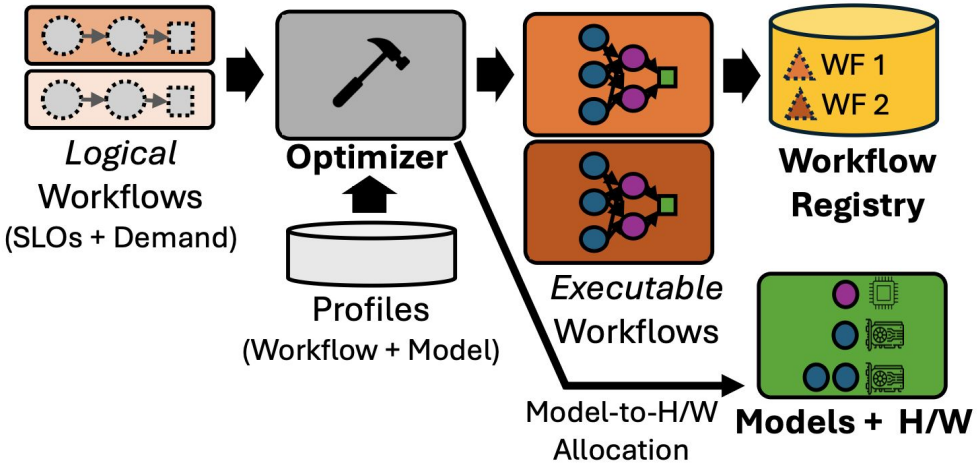


Methodology



- Parse task descriptions and data dependencies
- Match each task to an executor from the library
- Build a logical workflow as a DAG

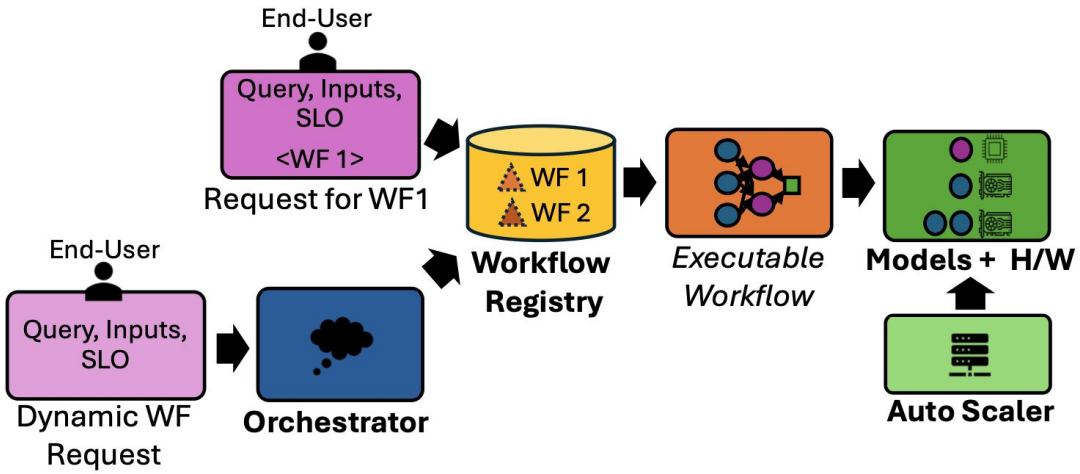
Methodology



(b) Optimization and deployment phase.

- Use workflow and model profiles
- Select workflow, model, and hardware configurations based on SLOs and demand
- Generate executable workflows and model to hardware allocation

Methodology



(c) Execution phase.

- Supports both registered and dynamic workflow requests
- Retrieve and execute the SLO-specific workflow
- Adapt model instances with auto-scaling

Test setup and Evaluation

Video Q/A + code generation

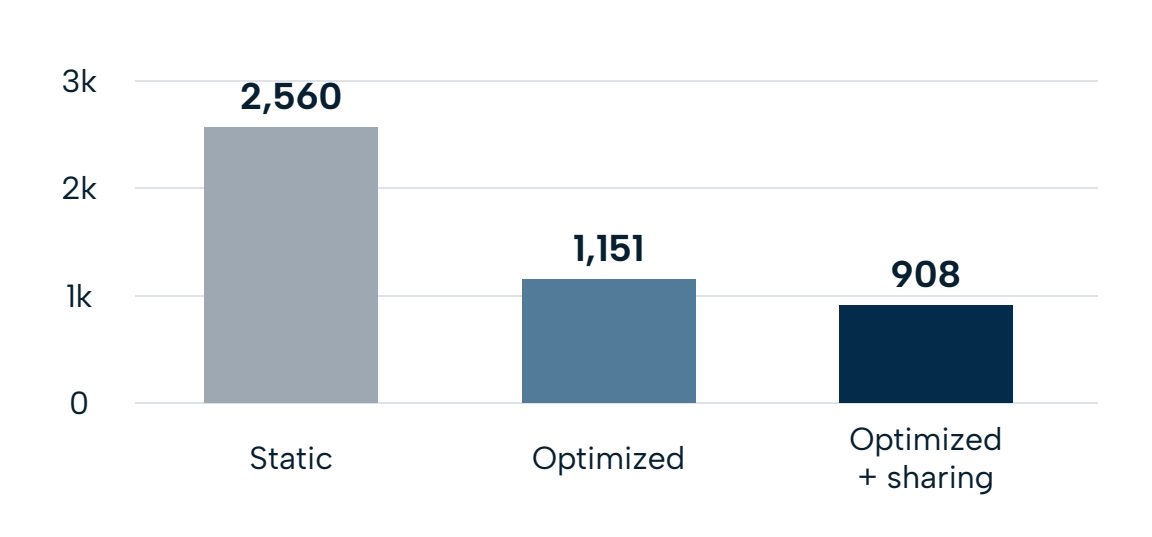
24-hour Azure LLM trace used as an arrival proxy
70% high-accuracy requests / 30% low-latency requests

Three policies

Static: fixed, hand-tuned configuration
Optimized: adapt each workflow–SLO combination
Optimized + sharing: jointly share model instances

Murakkab reduces GPU use by 65%

GPUs across the 24-hour workload



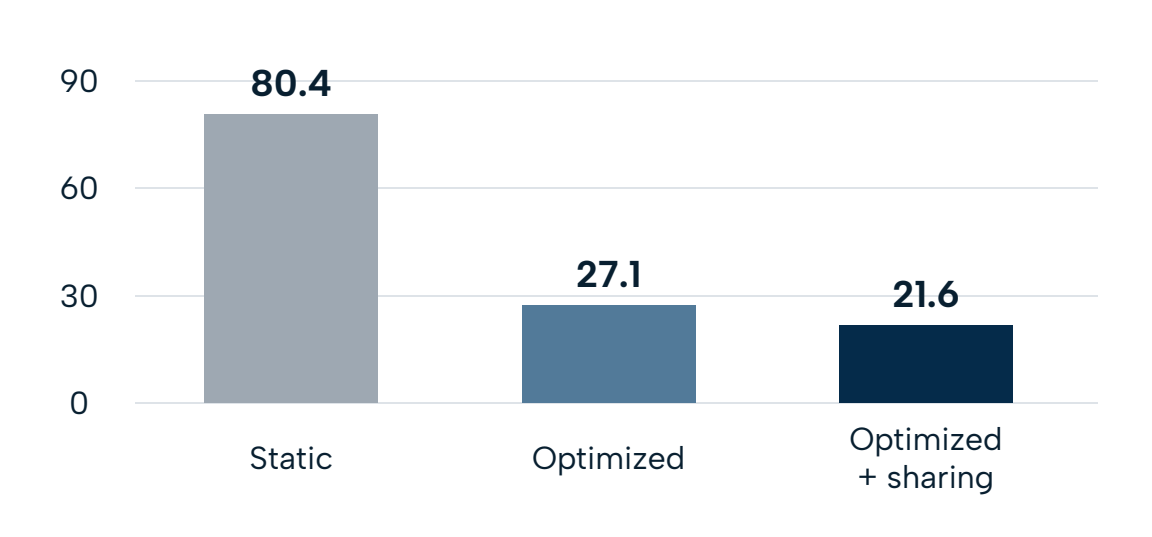
2.8× fewer GPUs

Sharing adds a further 21% reduction.

Source: paper, Table 1. Same workload and request SLO mix.

Murakkab reduces energy use by 73%

Energy (MWh) across the 24-hour workload



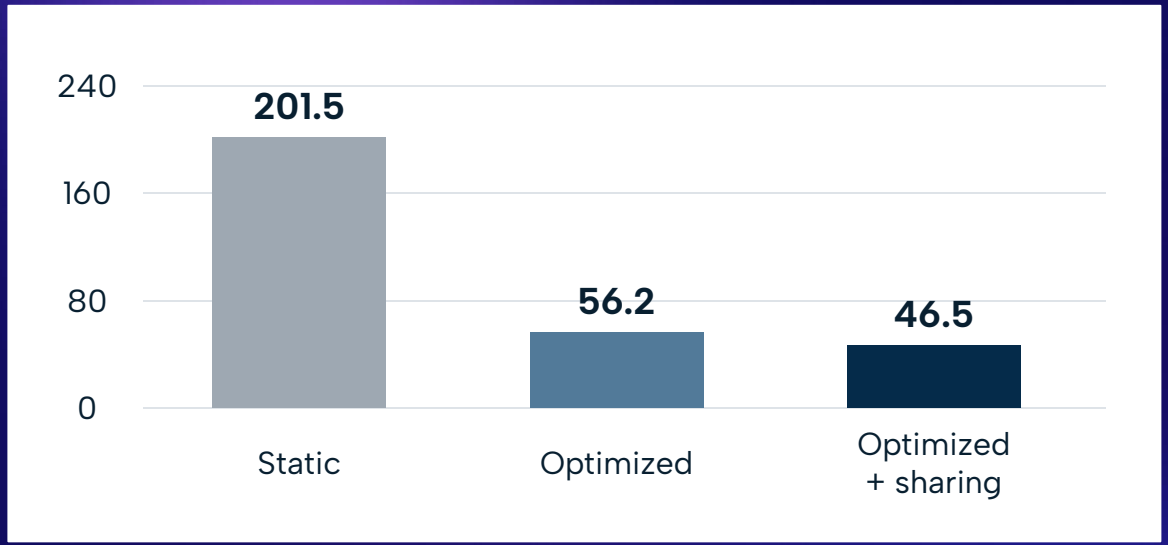
3.7× less energy

80.4 MWh falls to 21.6 MWh.

Source: paper, Table 1. Same workload and request SLO mix.

Murakkab reduces serving cost by 77%

Cost ($ thousands) across the 24-hour workload



4.3× lower cost

$201.5K falls to $46.5K.

Source: paper, Table 1. Same workload and request SLO mix.

Results: the trade-off

More H100s reduce energy, but can raise cost

H100s available	Energy (MWh)	Cost ($K)
0	24.7	55.7
500	11.0	75.4

The optimizer follows the chosen objective and SLOs.

Source: paper, Table 2. Energy minimization with 2,000 A100s available.

Takeaways and discussion

Efficiency gains while meeting the selected SLOs

Takeaway: coordinate configuration and sharing to meet SLOs efficiently.
Energy and cost can conflict.

Limits: two main workflows and proxy arrivals.
How robust are choices when profiles or workloads change?

Discuss: who should choose the energy–cost trade-off?
The reported savings remain workload-specific.

Thank you

The floor is open for discussion